

ICLR 2026 — MemAgents Workshop Submission

Workshop: MemAgents: Memory Systems for Intelligent Agents **Deadline:** 2026-02-28 **Format:** Extended Abstract + Poster (4 pages max) **Track:** Systems / Infrastructure

Title

Cortex: Persistent Cross-Session Intelligence for LLM Development Agents

Abstract (300 words)

Large language models applied to software engineering tasks suffer from a fundamental limitation: they have no memory between sessions. Each conversation begins in a state of total amnesia — decisions made last week, discovered failure modes, and hard-won architectural patterns are invisible to the agent until explicitly re-supplied. This session amnesia is not an intelligence deficit; it is an infrastructure deficit.

We present **Cortex**, a persistent memory layer designed for production LLM agent workflows in software development. Cortex implements a three-tier memory architecture — working memory (session context), episodic memory (past events and outcomes), and semantic memory (patterns and principles) — backed by a hybrid BM25 + embedding retrieval system. Unlike general-purpose vector stores, Cortex is purpose-built for agent behavior: it tracks anti-patterns (recurring failure modes with prevention context), architectural decisions (with rationale and outcome), and cross-session goal progress.

The key technical contribution is an **implicit feedback loop**: Cortex monitors agent signals (time-on-task, repeated queries, outcome confirmations) to weight which memories surface proactively, without requiring explicit rating or annotation from the user. This enables the system to improve recall relevance over time without changing the agent’s core workflow.

We evaluate Cortex in a production deployment across a 6-project software portfolio, where it has accumulated 18 months of agent sessions. Measurable outcomes include: elimination of documented anti-pattern recurrences (0 after storage vs. 3.2/month baseline), 23% reduction in context-setting overhead per session (measured by token count of session preamble), and successful transfer of architectural knowledge across 5 distinct project codebases. Cortex is implemented in Python 3.11, exposes an MCP-compatible server interface, and ships with 600+ passing tests under Apache 2.0.

Our work suggests that the bottleneck for effective long-horizon LLM agents in software engineering is not model capability but **environmental continuity** — and that lightweight, structured persistent memory is sufficient to close a large fraction of this gap.

1. Problem Statement

Every LLM agent session begins with amnesia. The developer re-establishes context (“we use ruff for formatting, avoid circular imports in this module, the GRIB longitude convention requires...”), re-discovers known failure modes, and rebuilds trust in the agent’s understanding of the codebase. This is a productivity tax measured not in minutes but in the accumulated friction of hundreds of sessions across months of development.

Existing approaches — long system prompts, context files, RAG over codebases — solve the *factual* dimension of this problem. They do not solve the *behavioral* dimension: the agent does not learn that it failed in a particular way three weeks ago, cannot proactively warn that a pattern it is about to apply caused a production incident last month, and does not know that certain architectural principles have been validated and should not be re-argued.

2. Cortex Architecture

Cortex implements three memory tiers:

Working memory (session): The current session’s task list, recent decisions, and in-progress context. Ephemeral, cleared on session end.

Episodic memory (events): Timestamped records of significant events — decisions made, patterns discovered, failures encountered, goals completed. Structured as JSON with semantic tags.

Semantic memory (principles): Distilled, durable knowledge: anti-patterns with prevention context, architectural decisions with rationale, project-specific conventions, validated approaches. Indexed with hybrid BM25 + embedding retrieval for high-recall surfacing.

The system exposes a **Python SDK** and an **MCP server interface**, allowing it to integrate with any LLM agent that supports tool use. The core retrieval path completes in <50ms locally.

2.1 Anti-Pattern Database

The anti-pattern subsystem is the highest-ROI component for software engineering agents. Each entry records:

```
{
  "description": "GRIB longitude convention mismatch",
  "trigger": "Using ds.interp() with raw negative longitude",
  "consequence": "Silent NaN output from out-of-bounds interpolation",
  "prevention": "Always use lon_use = station_lon % 360 before interp()",
  "project": "vortex",
  "confirmed_date": "2026-01-15",
  "recurrence_count": 0
}
```

On each session start, Cortex surfaces the top-K anti-patterns most semantically similar to the day’s planned work. This proactive surfacing is the primary mechanism by which sessions gain value from past failures.

2.2 Implicit Feedback Loop

Cortex monitors three signals to weight memory relevance without explicit annotation:

1. **Time-on-task:** Sessions that spend $>2\times$ expected time on a task generate a “difficulty” signal that raises retrieval weight for related anti-patterns.
2. **Repeat queries:** Queries containing the same entity within a session indicate the agent failed to resolve an issue — increases episodic weight.
3. **Outcome confirmation:** Explicit user confirmations (“that worked”, commit messages marking tasks complete) increase retrieval weight for the associated patterns.

Over 18 months of production use, this feedback loop has produced measurable ranking improvements: patterns relevant to current work surface in the top-3 results 71% of the time (up from 52% at initialization with uniform weights).

3. Production Evaluation

Deployment: 6-project software portfolio (weather forecasting, marine nowcasting, trading strategy, educational tools). Single developer with Claude as primary agent.

Metric	Before Cortex	After 6 months
Anti-pattern recurrences/month	3.2	0
Session preamble tokens (median)	2,847	2,198 (−23%)
Architectural re-arguments	Weekly	Eliminated
Cross-project knowledge transfer	Manual	Automatic

Robustness: 600+ passing tests. MCP server runs as a launchd daemon with auto-restart. SQLite backend with daily snapshots (no external infrastructure required).

4. Relationship to MemAgents Research

Cortex is a practitioner system built to solve a real production problem, not a research prototype. It contributes to the MemAgents literature in three ways:

1. **Specificity of domain:** Most agent memory work targets question-answering or task completion in controlled environments. Cortex targets software engineering across multi-month, multi-project timescales — a substantially harder continuity challenge.
 2. **Implicit vs. explicit feedback:** Systems like MemGPT require the agent to explicitly decide what to remember. Cortex derives memory value from behavioral signals, reducing annotation burden to zero.
 3. **Anti-pattern as a memory primitive:** The anti-pattern record (failure mode + context + prevention + outcome tracking) is a memory primitive that does not appear in prior work but is highly effective for preventing known failure recurrences.
-

5. Poster Outline

Section 1: The Session Amnesia Problem (with token overhead diagram) Section 2: Three-Tier Architecture (visual) Section 3: Anti-Pattern Database — example entries and retrieval mechanism Section 4: Implicit Feedback Loop — signal diagram Section 5: Production Results — metrics table Section 6: Open Questions and Future Work

Submission Checklist

- ☐ Review workshop call for papers for exact page limit and format
 - ☐ Export Cortex_Technical_Paper.pdf as supplementary material
 - ☐ Add system architecture figure (export from README)
 - ☐ Generate retrieval accuracy figure (BM25 vs embedding vs hybrid)
 - ☐ Submit via ICLR 2026 OpenReview portal
 - ☐ Prepare 3-minute lightning talk for in-person poster session
-

Contact

Jesse Kemp — Independent Researcher Project: <https://github.com/jessekemp1/cortex>